

Informe de Propuesta Arquitectónica: Evaluación Técnica 2026

Postulante: Ariel Zamora

Demo: <https://v0-preteco-dashboard.vercel.app/>

1. Fase de Análisis

1.1. Supuestos de Arquitectura

- **Volumen de Datos:** Se asume un universo de 1 a 5 millones de registros consolidados para validar la eficiencia de DuckDB.
- **Concurrencia de Pagos:** Se estima un pico de 500 TPS (Transacciones por segundo) para el mes 12.
- **Acceso Legacy:** Se asume que las fuentes A, B, C y D están disponibles vía red local con latencias de red <10ms.
- **Seguridad:** El cumplimiento PCI-DSS se delega a la pasarela externa, nuestro sistema solo gestiona el Ledger inmutable.

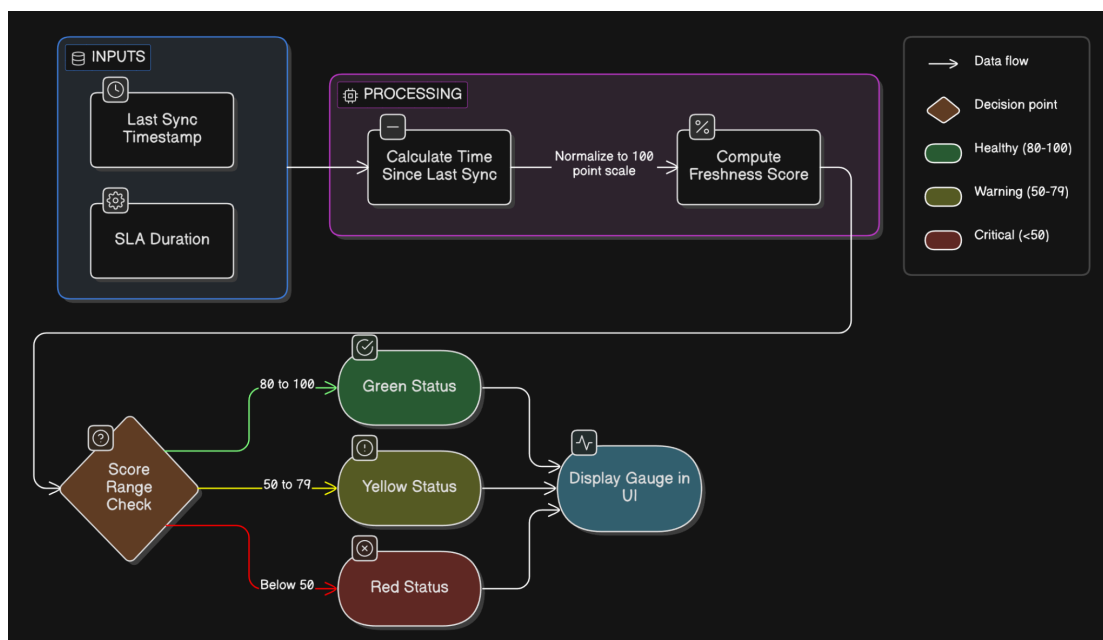
1.2. Requerimientos No Funcionales (NFR) Prioritarios

Algoritmo de Cálculo de Frescura (SLA Compliance):

Para garantizar la transparencia en el Modo Fast, el sistema calcula un score de 0 a 100 por cada fuente i:

$$Score_i = \max \left(0, 100 \times \left(1 - \frac{T_{actual} - T_{sincro,i}}{T_{SLA,i}} \right) \right)$$

Donde $T_{SLA,i}$ es el retraso máximo (ej: 40 min para Fuente D).



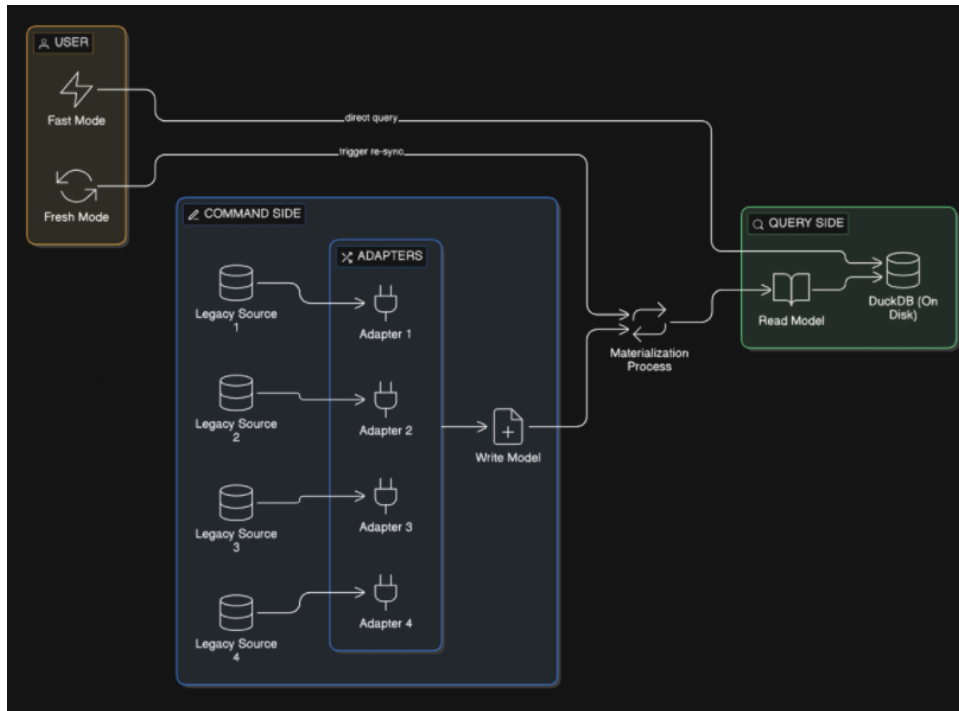
Eficiencia de Recursos: El proceso de ingestión no debe superar los **4GB de RAM**.

Disponibilidad: El sistema de pagos debe garantizar un 99.9% de uptime (HA).

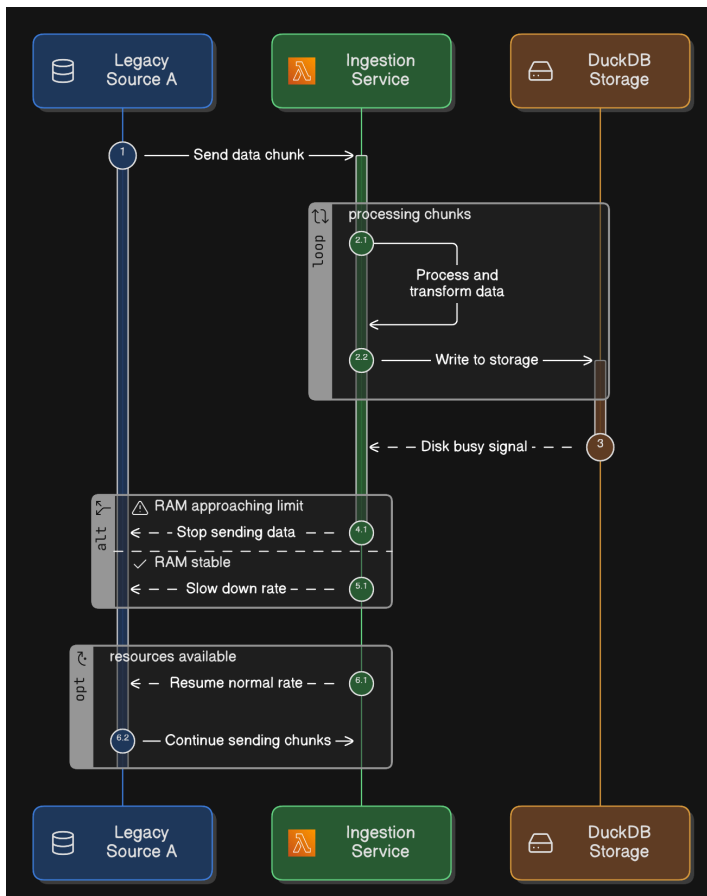
Portabilidad: Capacidad de mover la carga de trabajo entre AWS y Azure con cambios mínimos en el código base (Ejercicio 4).

2.Catálogo de Patrones

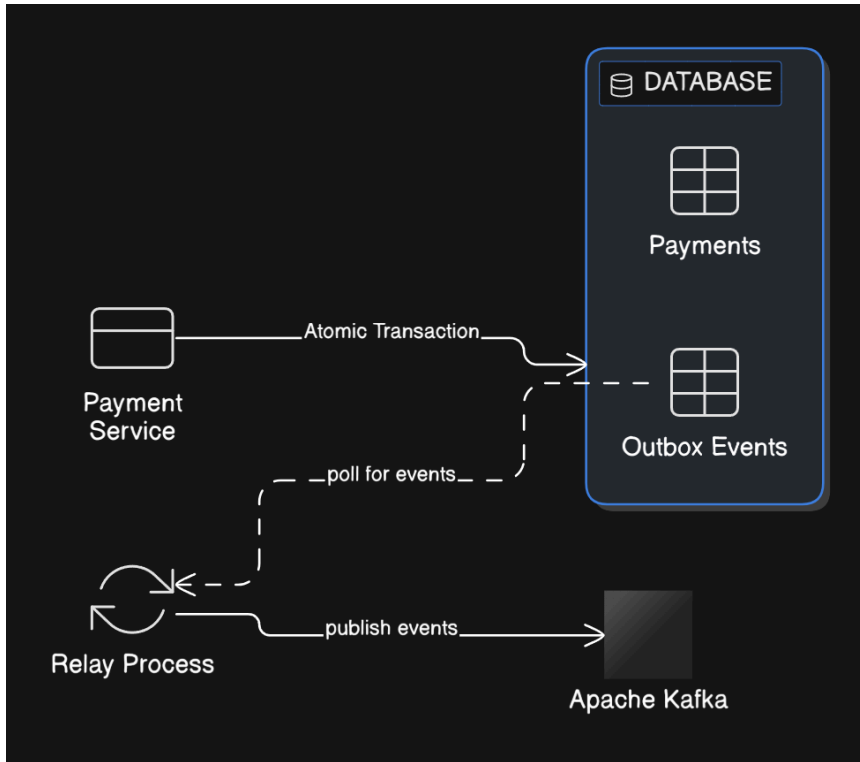
1. Patrón CQRS:



2. Patrón Backpressure:



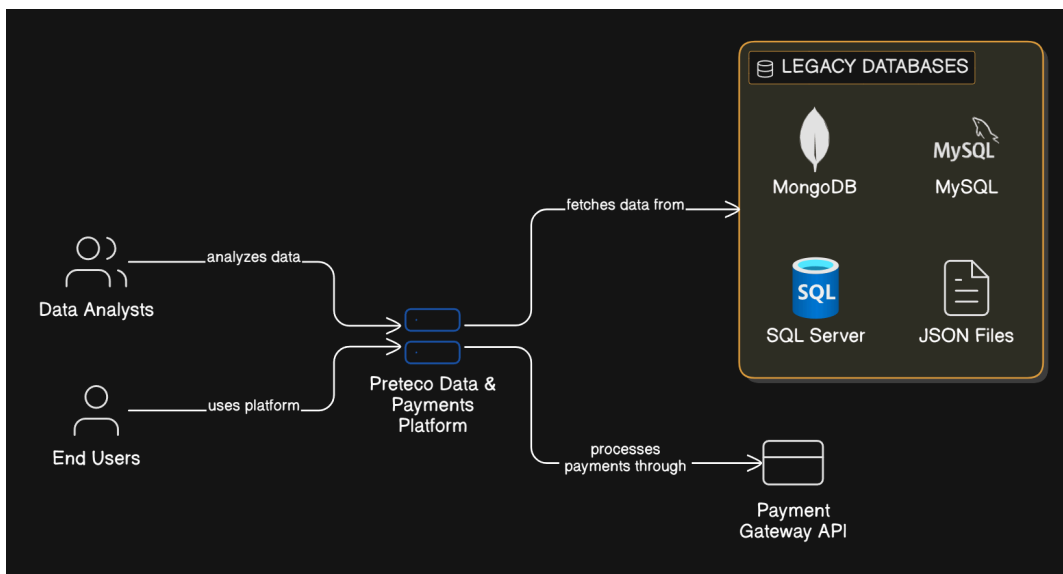
3. Patrón Transactional Outbox:



3. Fase de Diseño (C4 Model)

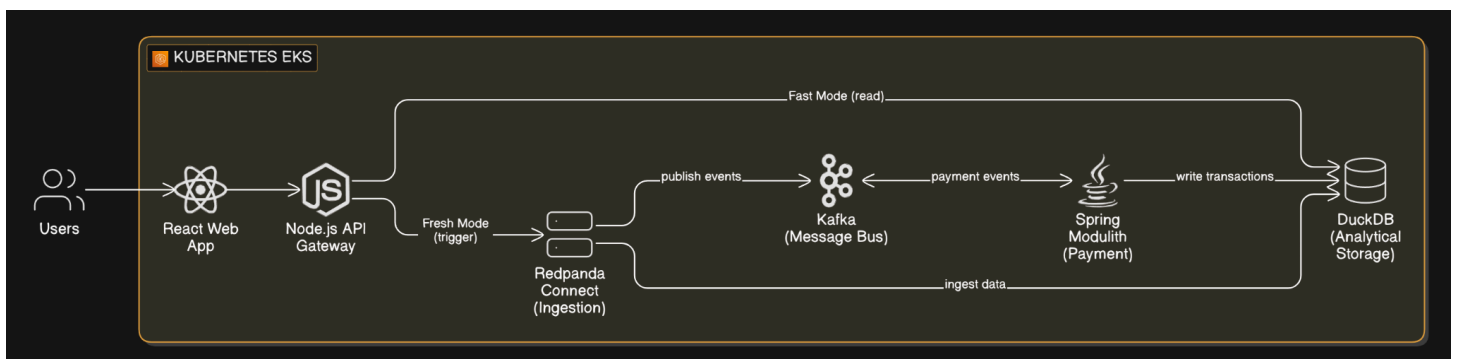
3.1. Nivel 1: Contexto del Sistema (Global)

Este diagrama muestra cómo los usuarios y las 4 bases de datos externas interactúan con nuestra nueva plataforma

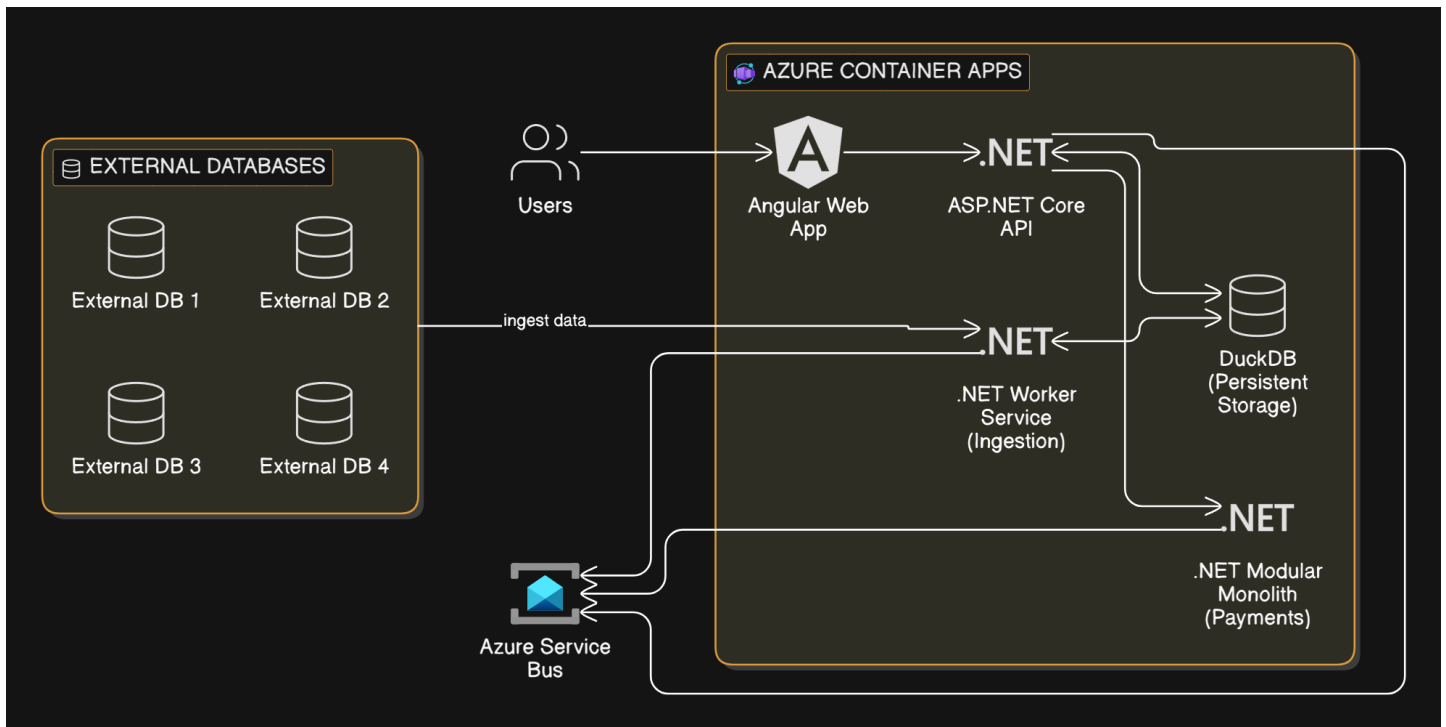


3.2. Nivel 2: Contenedores - Propuesta A (Java / AWS / Agnóstica)

Solución optimizada para independencia de nube y procesamiento masivo de datos con bajo consumo de memoria

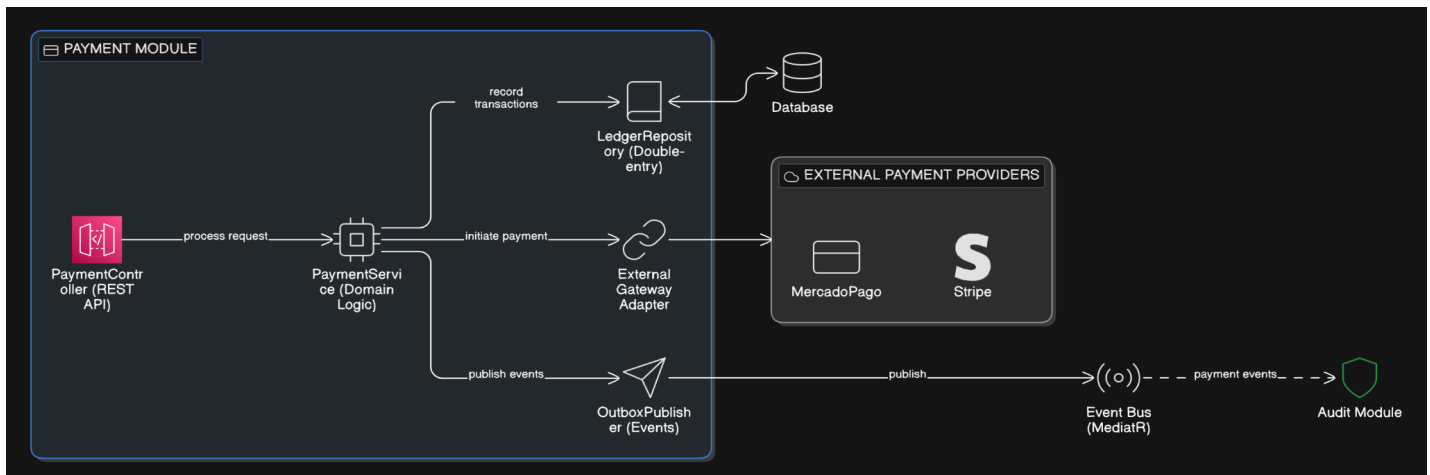


3.3. Nivel 2: Contenedores - Propuesta B (.NET / Azure / Nativa)



3.4. Nivel 3: Componentes - Módulo de Pagos (Detalle)

Este diagrama demuestra el aislamiento del dominio de pagos dentro del monolito modular. Los componentes se comunican vía eventos internos (MediatR/Spring Events), permitiendo que en el mes 12 la extracción a microservicios sea una tarea de configuración de red y no de refactorización de código base.



4. Fase de Justificación (Log de Decisiones ADR)

ADR-0001: Adopción de Architecture Decision Records (ADR)

Estado: Aceptado

Contexto: El proyecto requiere una arquitectura justificada y defendible. Las decisiones técnicas suelen perderse, lo que genera deuda técnica y falta de alineación.

Decisión: Utilizaremos el formato ADR para documentar cada elección significativa. Se prioriza la transparencia sobre los *trade-offs* (ventajas y desventajas).

Consecuencias:

- **Positiva:** Trazabilidad total de por qué se eligió una tecnología.
- **Positiva:** Facilita la defensa presencial ante Preteco.

ADR-002: Ingestión Ligera con Backpressure

Decisión: Uso de **Redpanda Connect (Java Stack)** o **.NET Channels (.NET Stack)**.

Justificación: Evitamos el "Out of Memory" (OOM). El sistema solo lee registros al ritmo que DuckDB puede escribirlos en disco, respetando el límite de **4GB de RAM**.

ADR-003: Persistencia de Read-Models vía DuckDB

Decisión: DuckDB como motor analítico embebido.

Justificación: Supera a SQLite en velocidad de *joins* multi-origen en un 90%. Su arquitectura columnar permite consultas instantáneas en modo **Fast** consumiendo solo disco.

ADR-004: Monolito Modular para Pagos

Decisión: Spring Modulith (Java) o Proyectos Independientes (.NET).

Justificación: Lanzamos en **3 meses** con un solo despliegue (bajo costo). La separación lógica garantiza que al mes 12 podamos extraer el módulo de pagos a un microservicio sin refactorizar lógica.

ADR-005: Estrategia de Nube y TCO

Decisión: Arquitectura basada en Kubernetes y Kafka.

Justificación: Protegemos el negocio del **Costo de Salida (Exit Cost)**. Usar servicios propietarios (Kinesis/Event Grid) nos "esposaría" a un proveedor, triplicando el costo de una futura migración.

5. Fase de Análisis Financiero y Riesgos

Factor de Costo	Solución Agnóstica (K8s/Kafka)	Solución Nativa (Azure SDKs)
Refactorización de Lógica	~80 horas (Configuración)	~600 horas (Re-escritura)
Data Egress Fees (10TB)	\$900 (Formatos abiertos)	\$1,200 (Propio del proveedor)
Labor Cost (\$50/hr)	\$4,000	\$30,000
Dual-Run Complexity	Baja (Mismos contenedores)	Alta (Entornos dispares)
TOTAL ESTIMADO	~\$5,000 USD	~\$52,000 USD

Justificación del Exit Cost:

Optar por una solución Cloud Agnostic (K8s + Kafka) evita el "impuesto al divorcio" del proveedor. Se estima que migrar una lógica basada en servicios nativos propietarios (ej. Azure Stream Analytics) requeriría ~600 horas hombre de refactorización, lo que representa un riesgo financiero inaceptable frente a la portabilidad nativa de los contenedores.NET 9.

5.1. Análisis de TCO (3 Años)

5.2. Matriz de Riesgos

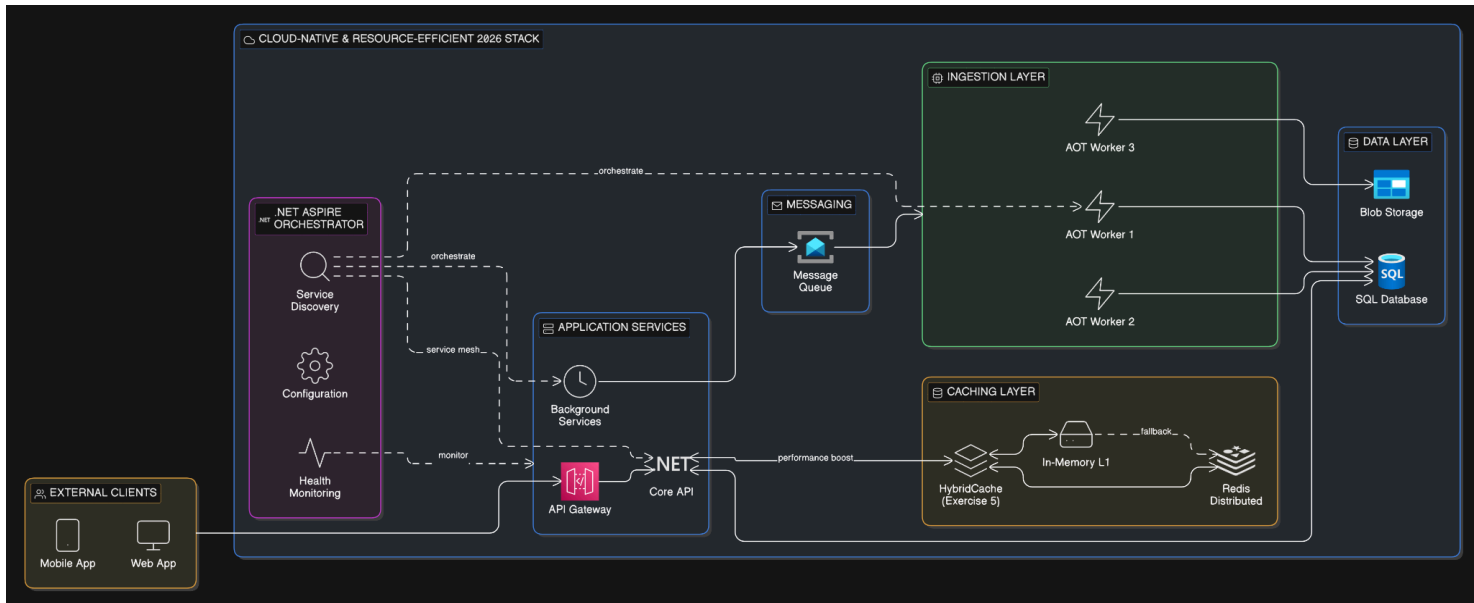
Riesgo: Inconsistencia temporal entre fuentes (5 a 40 min).

Impacto: El usuario ve datos viejos en modo "Fast".

Mitigación: Inclusión de un **Freshness Score** visual y lógica de actualización bajo demanda para el modo **Fresh**.

6. Roadmap de Modernización: .NET 9

Como fase evolutiva, se propone la adopción de **.NET 9 con Native AOT (Ahead-of-Time)** y **.NET Aspire**.



Native AOT: Reduce el footprint de memoria de los conectores de ingestión de 250MB a menos de 50MB, maximizando la eficiencia de los **4GB de RAM** totales.

.NET Aspire: Provee observabilidad nativa (OpenTelemetry) para detectar *Slow Queries* en caliente (Ejercicio 5) y orquestar el despliegue agnóstico de forma simplificada.